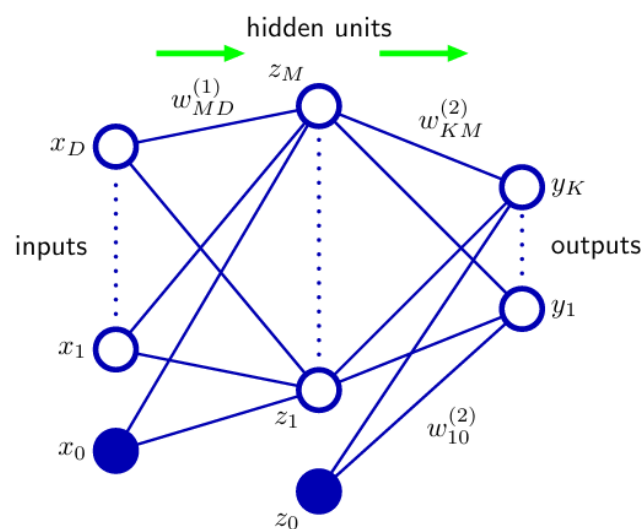


Project #2: Multi-layer Perceptron

Introduction

Recently, generative AI, such as Chat-GPT, has brought us to the new generation of life. In fact, generative AI is also a kind of machine learning methods, which is called deep learning. Here, we have a chance to peek into the world of deep learning, by implementing classical Muti-layer perceptron with back-propagation algorithms. Muti-layer perceptron, also known as MLP, is constituted by layers of neuron, together with some linear algebra functions to generate the output. After generating the first output, we should calculate the loss function to know how our MLP model is predicting good or not. By using 'Back-propagation algorithm', we can improve our model by updating its weights and bias through gradient. The aim of this project is to let you understand how neurons work to generate the result and understand how to train a neural network.



(Figure 5.1. *Pattern Recognition and Machine Learning. p. 231.)

Objectives

In this project, there's some jobs you need to finish. First, you need to use the code you wrote in project #1 to handle data preprocessing for project #2. Second, implement MLP class in `model.py/model.hpp` and train it on your own. Finally, evaluate your MLP model and use it to predict the outcome.

(Go check [*proj2_guildline_python.pdf*](#) or [*proj2_guildline_cpp.pdf*](#) for the further detail)

Dataset Description

The data for this project is real-word data for hospitalizing patients. Each patient is described by 77 *features* (F1~F77), together with a binary outcome. The project involves training model with provided datasets (`train_X.csv` & `train_y.csv`) and final testing on an independent test set (`test_X.csv` & `test_y.csv`). Detailed information on patient features is available in [*Data_Description.pdf*](#).

Note:

- *The distribution of the data in this project is different from the previous one.*
- *Some categorical features are different from the others.*

Language:

You can implement your code by using either *Python3* or *C++*.

(just need to submit one of them)

Requirements:

1. Download the necessary library:

This part is **same** as project #1.

2. Preprocess the data

Use the code you wrote in the previous project to handle data preprocessing.

(You may need to modify your code for handling different categorical values and select features from now on)

3. Implement the detail of MLP class:

- The **basic** requirement is to implement MLP with **two** hidden layers (1 input layer, 2 hidden layers, 1 output layers), **three** different activate functions, and **three** different optimizers
(For advance requirement, please check bonus part in report spec)
- Number of neurons in hidden layer can be any positive integer
- Each hidden layer needs to contain one transform layer (Linear) and one activate function (Sigmoid, Relu, and so on)
- Implement the model with and without activation function
- More than three activate functions and optimizers is fine
- Implement backpropagation algorithm in MLP class to calculate the gradients

4. Train your MLP model:

After you finish implementing your MLP model, please use `fit(X, y)` method to train the model.

(It is welcome for you to write your own training method but please call it in `fit(X, y)`. You can refer to `LogisticRegressionClassifier.fit()` in project #1 to see how I wrote training code for LR model in `fit(X, y)`)

5. Performance measuring:

Adjust the hyperparameter of MLP model, screenshot the best results and paste it in your report.

6. Do your implement **from scratch**

Do not use any machine learning libraries, except for the part we've already done for you in the framework.

Grading Policy (Total 110 pt includes bonus)

1. Report Spec (60%) (follow this format to write your report)

- A. Explain the concept and code detail of your MLP model, including *MLPClassifier*, *activation functions*, *optimizers* (30%)
- B. Please answer the following questions (20%)
 - 1. What is gradient explosion? If you encounter this problem, how do you solve it? Please be precise and concise. (10%)
 - 2. What is overfitting? If you encounter this problem, how do you solve it? Please be precise and concise. (10%)
- C. Discussion (10%)
 - 1. Compare the result with and without activate functions. Do you think activate function is necessary in neural networks? (4%)
 - 2. Compare the results of three different activate functions. E.g. Sigmoid, Relu, ... (3%)
 - 3. Compare the results of three different optimizers. E.g. SGD, momentum, ... (3%)
- D. Bonus (10%)

Implement general MLP model (n layers). That is, make your MLP model scalable so that it can scale to any depth and width. Then, Compare the result of different depths (e.g. depth= 2, 5, 10). Do you think a deeper MLP model is better in this project?

(The requirement is that one of your parameters in your constructor need to take a list or array-like shape ([*num_features*, 100, 100, 100, 1] or [*num_features*, 100, 200, 300, 400, 1]) as model structure)

```
model = mlpClassifier(layers=[77, 50, 100, 200, 100, 50, 1])
```

e.g. The screenshot shows that this MLP model has one input layer, five hidden layers, and one output layer. The value of each layer means the number of neurons in it.

2. Performance (25%)

For your result of performance, you'll get:

- 25/25: if scoring ≥ 0.63
- 20/25: if $0.6 > \text{scoring} \geq 0.55$
- 15/25: if $0.55 > \text{scoring} \geq 0.5$
- 0/25: if scoring < 0.5

$$\text{*Scoring} = 0.3 * \text{acc} + 0.35 * \text{f1 score} + 0.35 * \text{mcc}$$

Performance Metric	Definition
Recall ^a	$TP/(TP + FN)$
Precision ^b	$TP/(TP + FP)$
ACC	$(TP + TN)/(TP + TN + FP + FN)$
F1-score	$\frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}$
MCC	$\frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP) \times (TP + FN) \times (TN + FP) \times (TN + FN)}}$
AUC	Area under the ROC curve

TP, true positive; TN, true negative; FP, false positive; FN, false negative; MCC, Matthews correlation coefficient; ACC, accuracy; AUC, area under the curve; ROC, receiver operating characteristic.

^a Recall is equivalent to sensitivity in its definition.

^b Precision is equivalent to positive predictive value in its definition.

Screenshot and **paste** your result as how the following picture looks like in your report.

```
Acc: 0.80233
F1 score: 0.70175
MCC: 0.55626
Scoring: 0.68100
```

Hint: Because most of you didn't do feature selection in project #1, here is a friendly suggestion that you may need to select features in this project, or your performance may not pass 0.63 if you only

adjust hyperparameters of the MLP model. (The reason is because some features may be irrelevant, redundant to the model. If you're interested in it, you can do some experiments on project #1 to see how features affect performance of the model)

Link: [A short Guild for Feature Engineering and Feature Selection](#)

*(As calling machine learning library is **prohibited**, you can implement feature selection function by referring to the code in it. Remember to include the reference in your report.*

If you want to use wrapper method to select features, you can use Logistic Regression model I provided in project #1 as feature selection model)

3. Source code (15%)

We will check your code to see if your code is implemented like what you explain in part 1-A of your report. Then, **we'll run your code** by using the raw data we gave you on E3 to make sure you do not cheat in the result.

If you're worried that we cannot run your code properly, you can explain how to run your code in detail in your report.

e.g. Tell us how to execute your code and where we should modify the path.

Submission Deadline:

Deadline: 10/27 (Sun.) 11:59 p.m.

Submission format:

- You need to submit **all your code in one folder** together with your **report file**, and zip them into one **.zip** file.
- Name your code folder as “<language>_hw2_<yourStudentID>_<yourName>”,
e.g. “Cpp_hw2_312551120_陳柏憲” or “Python_hw2_312551120_陳柏憲”
- Name your report file as “report_hw2_<yourStudentID>_<yourName>.pdf”,
e.g. “Report_hw2_312551120_陳柏憲.pdf”
- Name your .zip file as “ml_hw2_<yourStudentID>_<yourName>.zip”,
e.g. “ML_hw2_312551120_陳柏憲.zip”

(Uppercase or lowercase of the letters is fine)

Punishment Policy:

Except for what we mentioned in this file, please refer to *project_policy.pdf* we provided in project #1.

Also, in this project, if your performance metrics are very similar to your classmate's, we may suspect cheating and check your code very carefully. Maybe we will ask you and your classmate to explain it in person. Please be aware!

Good luck~

TAs